TECHNICAL PROJECT REPORT

Secure Hybrid Kubernetes Delivery

Crewmeister: DevSecOps & GitOps Implementation

Spring Boot • VMware Kubernetes • Azure AKS • Infrastructure as Code • Observability

1. Context and engineering objectives

Crewmeister application. The exercise centers on a Java Spring Boot REST API backed by MySQL. The target is a reproducible delivery path from source code to containers, Kubernetes deployments, and operational evidence, across a local lab and Azure-managed infrastructure.

Hybrid DevOps architecture. The design uses one common application packaging model while retaining environment-specific infrastructure: **three VMware/Ubuntu virtual machines** for an on-premises Kubernetes cluster, and an **Azure Kubernetes Service (AKS)** cluster provisioned with Terraform.

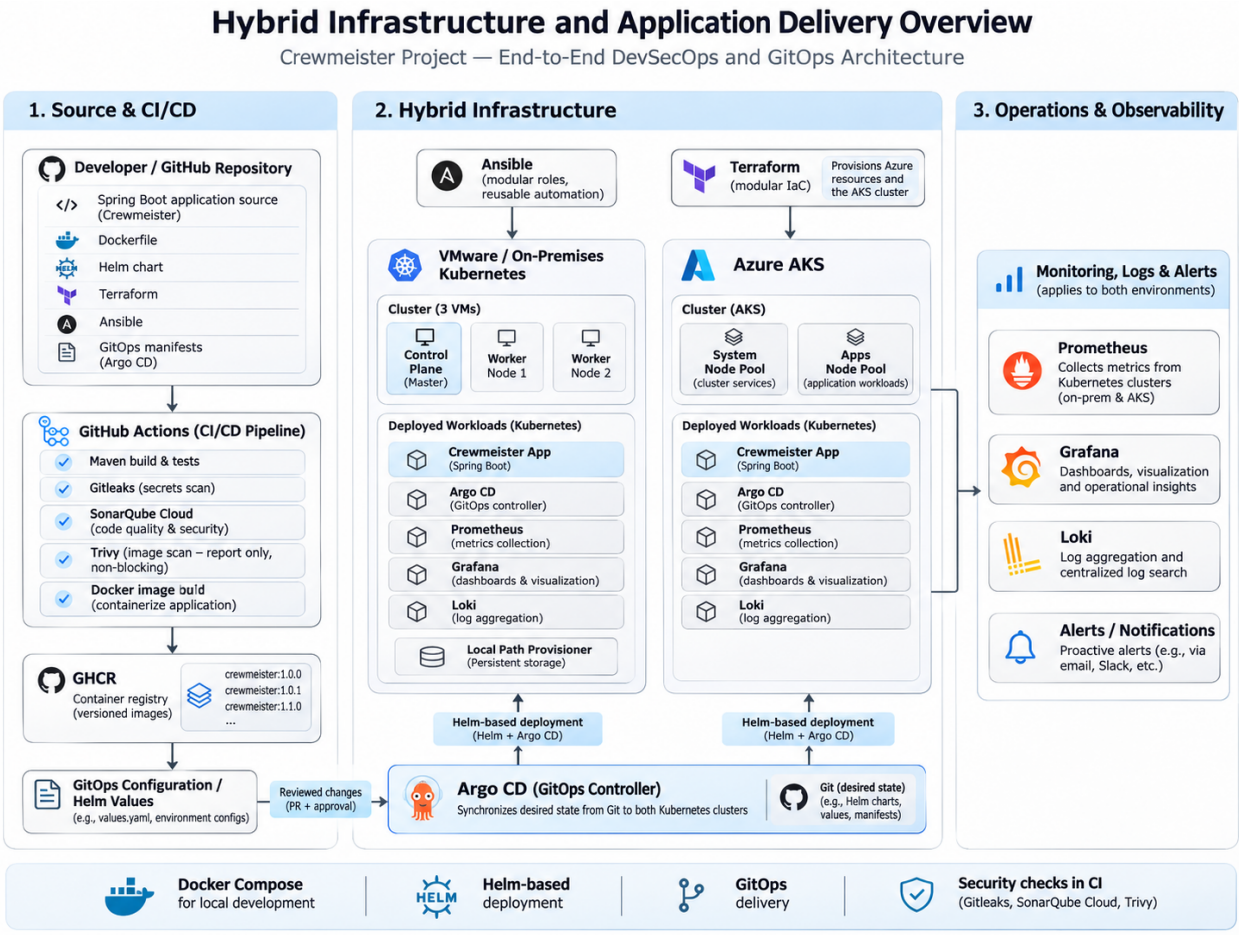


Figure 1: Hybrid infrastructure and application delivery overview, as documented in the repository.

Delivery method and scope

The delivery workflow is organized as **infrastructure automation → container build → CI and security feedback → GitOps change → Argo CD reconciliation → monitoring**. The project emphasizes modular IaC, reuse of the Helm chart, reviewable configuration, and observable operations.

WORKSTREAM 01 / ON-PREMISES INFRASTRUCTURE

2. Three-node Kubernetes on VMware

VMware and Ubuntu. VMware hosts a reproducible local lab of **three Ubuntu VMs**: one Kubernetes control-plane node (master) and two worker nodes. The control plane manages API and scheduling services; the workers execute application and platform workloads.

Kubernetes. Kubernetes provides declarative deployment, service discovery, and workload scheduling. The local cluster offers a second delivery target alongside AKS, making portability and failure diagnosis directly testable.

VM role	Purpose	Validation evidence
Control plane	Kubernetes control services	`kubectl get nodes`
Worker 1	Application and platform workloads	Node Ready; pods scheduled
Worker 2	Application and platform workloads	Node Ready; pods scheduled

```
bechi@bechir:~/crewmeister-app$ KUBECONFIG="$HOME/.kube/crewmeister-admin.conf" kubectl get nodes -o wide
NAME                                STATUS   ROLES           AGE     VERSION   INTERNAL-IP      EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION            CONTAINER-RUNTIME
k8s-worker1                         Ready    <none>          4d20h   v1.36.5   192.168.88.131   <none>        Ubuntu 22.04.4 LTS   6.8.0-138-generic (amd64)   containerd://2.2.1
k8s-worker2                         Ready    <none>          4d20h   v1.36.5   192.168.88.133   <none>        Ubuntu 22.04.4 LTS   6.8.0-138-generic (amd64)   containerd://2.2.1
mastercrewmeister-virtual-machine   Ready    control-plane   4d20h   v1.36.5   192.168.88.132   <none>        Ubuntu 22.04.4 LTS   6.8.0-138-generic (amd64)   containerd://2.2.1

bechi@bechir:~/crewmeister-app$ kubectl get pods -A -o wide
NAMESPACE             NAME                                                                READY   STATUS    RESTARTS          AGE     IP               NODE
argocd                argocd-application-controller-0                                     1/1     Running   0                 2d21h   10.244.126.53    k8s-worker2
argocd                argocd-applicationset-controller-d885ccf8d-s274d                    1/1     Running   0                 3d3h    10.244.126.48    k8s-worker2
argocd                argocd-dex-server-cc998dd89-8dc85                                   1/1     Running   0                 3d3h    10.244.126.46    k8s-worker2
argocd                argocd-notifications-controller-7955bfff9-szct8                     1/1     Running   0                 3d3h    10.244.126.47    k8s-worker2
argocd                argocd-redis-c74779d75-sqm6x                                        1/1     Running   0                 3d3h    10.244.126.44    k8s-worker2
argocd                argocd-repo-server-6c8569c7b4-7fr52                                 1/1     Running   1 (35h ago)       3d3h    10.244.126.45    k8s-worker2
argocd                argocd-server-7c778758c8-sgftv                                      1/1     Running   2 (35h ago)       3d3h    10.244.126.52    k8s-worker2
crewmeister           crewmeister-app-7c45f4dc7b-9v6b9                                    1/1     Running   1 (6h46m ago)     35h     10.244.194.111   k8s-worker1
crewmeister           crewmeister-app-7c45f4dc7b-rdbnk                                    1/1     Running   1 (35h ago)       45h     10.244.126.59    k8s-worker2
crewmeister           crewmeister-mysql-0                                                 1/1     Running   0                 2d19h   10.244.126.58    k8s-worker2
kube-system           calico-kube-controllers-7848b846c6-szcrp                            1/1     Running   4 (21h ago)       4d20h   10.244.178.7     mastercrewmeister-virtual-machine
kube-system           calico-node-fzcmj                                                   1/1     Running   2 (43h ago)       4d20h   192.168.88.132   mastercrewmeister-virtual-machine
kube-system           calico-node-jnb7t                                                   1/1     Running   3 (3d17h ago)     4d20h   192.168.88.131   k8s-worker1
kube-system           calico-node-wzfj5                                                   1/1     Running   2 (3d18h ago)     4d20h   192.168.88.133   k8s-worker2
kube-system           coredns-589f44dc88-8l5j6                                            1/1     Running   4 (21h ago)       4d20h   10.244.178.8     mastercrewmeister-virtual-machine
kube-system           coredns-589f44dc88-k5jt4                                            1/1     Running   4 (21h ago)       4d20h   10.244.178.9     mastercrewmeister-virtual-machine
kube-system           etcd-mastercrewmeister-virtual-machine                              1/1     Running   2 (43h ago)       4d20h   192.168.88.132   mastercrewmeister-virtual-machine
kube-system           kube-apiserver-mastercrewmeister-virtual-machine                    1/1     Running   4 (35h ago)       4d20h   192.168.88.132   mastercrewmeister-virtual-machine
kube-system           kube-controller-manager-mastercrewmeister-virtual-machine           1/1     Running   7 (22h ago)       3d15h   192.168.88.132   mastercrewmeister-virtual-machine
kube-system           kube-proxy-4dp4h                                                    1/1     Running   2 (43h ago)       4d20h   192.168.88.132   mastercrewmeister-virtual-machine
kube-system           kube-proxy-dxz62                                                    1/1     Running   3 (3d17h ago)     4d20h   192.168.88.131   k8s-worker1
kube-system           kube-proxy-kkkl4                                                    1/1     Running   3 (3d18h ago)     4d20h   192.168.88.133   k8s-worker2
kube-system           kube-scheduler-mastercrewmeister-virtual-machine                    1/1     Running   9 (21h ago)       3d15h   192.168.88.132   mastercrewmeister-virtual-machine
kube-system           metrics-server-55c6b8755f-whvhb                                     1/1     Running   3 (35h ago)       40h     10.244.126.62    k8s-worker2
local-path-storage    local-path-provisioner-848dc9bd9b-d8579                             1/1     Running   1 (35h ago)       3d3h    10.244.126.43    k8s-worker2
monitoring            alertmanager-monitoring-kube-prometheus-alertmanager-0              2/2     Running   0                 2d21h   10.244.126.55    k8s-worker2
monitoring            alloy-675d96cc76-rhphg                                              2/2     Running   0                 40h     10.244.126.61    k8s-worker2
monitoring            loki-0                                                              2/2     Running   0                 35h     10.244.194.112   k8s-worker1
monitoring            loki-gateway-5fffbf5f6f-wr7b6                                       2/2     Running   1 (35h ago)       40h     10.244.126.60    k8s-worker2
monitoring            monitoring-grafana-54497b475f-tv6pl                                 3/3     Running   1 (35h ago)       35h     10.244.126.63    k8s-worker2
monitoring            monitoring-kube-prometheus-operator-699f6d7b6c-cvrcw                1/1     Running   3 (35h ago)       3d3h    10.244.126.51    k8s-worker2
monitoring            monitoring-kube-state-metrics-764584797b-fsbg5                      1/1     Running   9 (35h ago)       3d3h    10.244.126.50    k8s-worker2
monitoring            monitoring-prometheus-node-exporter-g2f6f                           1/1     Running   0                 3d17h   192.168.88.131   k8s-worker1
monitoring            monitoring-prometheus-node-exporter-l7pg9                           1/1     Running   1 (35h ago)       3d17h   192.168.88.133   k8s-worker2
monitoring            monitoring-prometheus-node-exporter-zpkg6                           1/1     Running   0                 70s     192.168.88.132   mastercrewmeister-virtual-machine
monitoring            prometheus-monitoring-kube-prometheus-prometheus-0                  2/2     Running   2 (35h ago)       2d21h   10.244.126.54    k8s-worker2
```

Figure 2: Validation of the on-premises Kubernetes cluster: one control plane and two worker nodes in Ready state.

Ansible: modular and reusable provisioning

Ansible. Ansible is an agentless automation framework. The project stores the cluster build as inventory, playbooks, shared variables, and reusable roles in `crewmeister-k8s-ansible/`; this reduces manual configuration drift and allows repeat execution.

Repository asset	Operational responsibility
`inventory.yml` / `group_vars/`	Node membership and cluster parameters
`preflight.yml` / `site.yml`	Prerequisite checks and orchestration
`roles/`	Reusable setup tasks and cluster components
`fetch-kubeconfig.yml`	Client access to the cluster
`reset-cluster.yml`	Intentional lab reset only

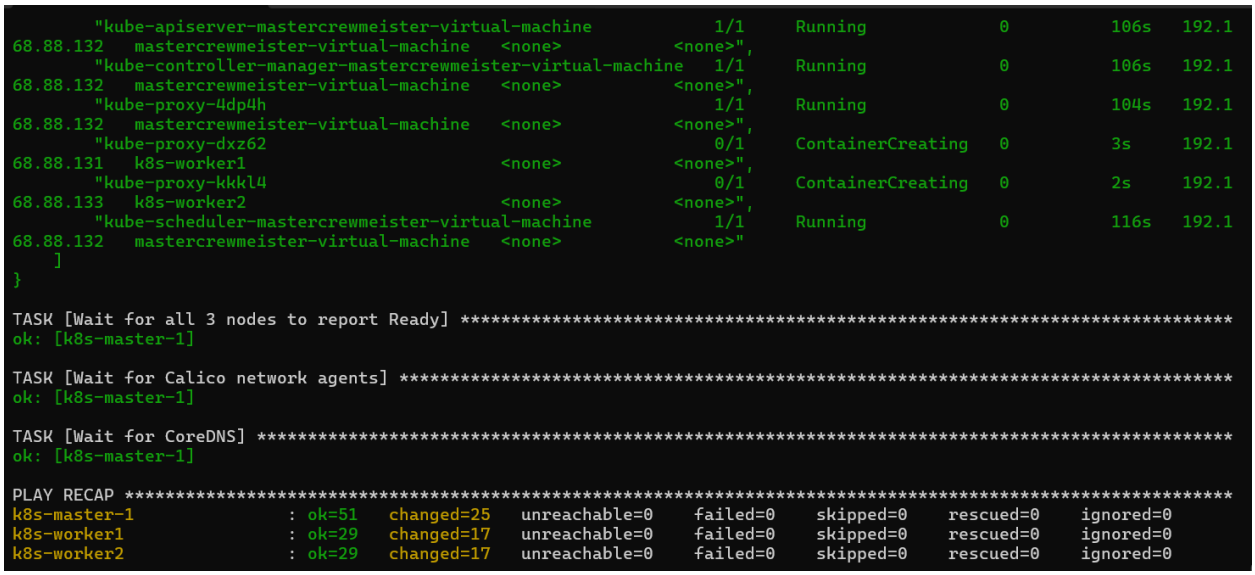


Figure 3: Ansible implementation evidence from the project screenshots.

Validation commands: `ansible-playbook -i inventory.yml site.yml -syntax-check`, followed by `kubectl get nodes -o wide` and `kubectl get pods -A`.

WORKSTREAM 02 / CLOUD INFRASTRUCTURE AS CODE

3. Azure AKS and modular Terraform

Azure Kubernetes Service. AKS provides a managed Kubernetes control plane and configurable worker capacity. This cloud environment complements the three-VM lab rather than replacing it.

Terraform. Terraform expresses Azure resources declaratively. The `terraform/` workstream separates provider/resource configuration from variables and reusable components. A plan exposes pending changes; an apply is required for actual resource creation.

Two distinct AKS node pools

The project architecture separates the **system pool**, used principally for Kubernetes platform services, from the **application (user) pool**, dedicated to application workloads. In the supplied node screenshot they appear as `aks-system` and `aks-apps`; this is a pool separation, not two separate clusters.

```
bechi@bechir:~/crewmeister-app$ kubectl --kubeconfig="$HOME/.kube/crewmeister-aks" get nodes -L agentpool
kubectl --kubeconfig="$HOME/.kube/crewmeister-aks" get pods -A -o wide
NAME                              STATUS   ROLES    AGE   VERSION   AGENTPOOL
aks-apps-40337985-vmss000000      Ready    <none>   16h   v1.36.5   apps
aks-system-27949052-vmss000000    Ready    <none>   22h   v1.36.5   system
NAMESPACE     NAME                                                READY   STATUS    RESTARTS      AGE   IP             NODE
argocd        argocd-application-controller-0                     1/1     Running   0             20h   10.244.0.220   aks-system-27949052-vmss000000
argocd        argocd-applicationset-controller-6649f4989f-m9g6g   1/1     Running   0             20h   10.244.0.226   aks-system-27949052-vmss000000
argocd        argocd-redis-86b594b897-c5bx7                       1/1     Running   0             20h   10.244.0.249   aks-system-27949052-vmss000000
argocd        argocd-repo-server-f7c764485-tqw6s                  1/1     Running   0             20h   10.244.0.176   aks-system-27949052-vmss000000
argocd        argocd-server-7c65b4b6bf-69czb                      1/1     Running   0             20h   10.244.0.17    aks-system-27949052-vmss000000
crewmeister   crewmeister-app-5d896f8d-7f4mg                      1/1     Running   2 (15h ago)   15h   10.244.1.231   aks-apps-40337985-vmss000000
crewmeister   crewmeister-mysql-0                                 1/1     Running   0             15h   10.244.1.8     aks-apps-40337985-vmss000000
kube-system   azure-cns-jr9ds                                     1/1     Running   0             16h   10.50.0.5      aks-apps-40337985-vmss000000
kube-system   azure-cns-qxnsp                                     1/1     Running   0             22h   10.50.0.4      aks-system-27949052-vmss000000
kube-system   azure-ip-masq-agent-48l9l                           1/1     Running   0             22h   10.50.0.4      aks-system-27949052-vmss000000
kube-system   azure-ip-masq-agent-br8tg                           1/1     Running   0             16h   10.50.0.5      aks-apps-40337985-vmss000000
kube-system   cilium-5n5nq                                        3/3     Running   0             22h   10.50.0.4      aks-system-27949052-vmss000000
kube-system   cilium-operator-857796698-dpbzh                     1/1     Running   0             22h   10.50.0.4      aks-system-27949052-vmss000000
kube-system   cilium-tb92h                                        3/3     Running   0             16h   10.50.0.5      aks-apps-40337985-vmss000000
kube-system   cloud-node-manager-4sgl4                            1/1     Running   0             16h   10.50.0.5      aks-apps-40337985-vmss000000
kube-system   cloud-node-manager-85pzs                            1/1     Running   0             22h   10.50.0.4      aks-system-27949052-vmss000000
kube-system   coredns-76df789c68-jc8jv                            1/1     Running   0             22h   10.244.0.129   aks-system-27949052-vmss000000
kube-system   coredns-76df789c68-m2f9c                            1/1     Running   0             22h   10.244.0.31    aks-system-27949052-vmss000000
kube-system   coredns-autoscaler-548c6d85c6-l9fgt                 1/1     Running   0             22h   10.244.0.141   aks-system-27949052-vmss000000
kube-system   csi-azuredisk-node-2bk87                            3/3     Running   0             16h   10.50.0.5      aks-apps-40337985-vmss000000
kube-system   csi-azuredisk-node-bcn9v                            3/3     Running   0             22h   10.50.0.4      aks-system-27949052-vmss000000
kube-system   csi-azurefile-node-2chjd                            4/4     Running   0             22h   10.50.0.4      aks-system-27949052-vmss000000
kube-system   csi-azurefile-node-w8s5l                            4/4     Running   0             16h   10.50.0.5      aks-apps-40337985-vmss000000
kube-system   konnectivity-agent-556bd8f6f9-kbvgh                 1/1     Running   0             22h   10.244.0.175   aks-system-27949052-vmss000000
kube-system   konnectivity-agent-556bd8f6f9-lv9tq                 1/1     Running   0             22h   10.244.0.241   aks-system-27949052-vmss000000
kube-system   konnectivity-agent-autoscaler-5b658f6697-jpp5v      1/1     Running   0             22h   10.244.0.190   aks-system-27949052-vmss000000
kube-system   metrics-server-dff79fd46-nlp9f                      2/2     Running   0             22h   10.244.0.244   aks-system-27949052-vmss000000
kube-system   metrics-server-dff79fd46-xgs4k                      2/2     Running   0             22h   10.244.0.97    aks-system-27949052-vmss000000
monitoring    alloy-698499448fb-4m7r8                             2/2     Running   0             18h   10.244.0.75    aks-system-27949052-vmss000000
monitoring    loki-0                                              2/2     Running   0             18h   10.244.0.136   aks-system-27949052-vmss000000
monitoring    monitoring-grafana-b8dc87d9-jhqbs                   3/3     Running   0             18h   10.244.0.160   aks-system-27949052-vmss000000
monitoring    monitoring-kube-prometheus-operator-699f6d7b6c-6cd45   1/1     Running   0             18h   10.244.0.235   aks-system-27949052-vmss000000
monitoring    monitoring-kube-state-metrics-764584797b-6j9v6      1/1     Running   0             18h   10.244.0.30    aks-system-27949052-vmss000000
monitoring    monitoring-prometheus-node-exporter-8gt6k           1/1     Running   0             18h   10.50.0.4      aks-system-27949052-vmss000000
monitoring    monitoring-prometheus-node-exporter-z5ctb           1/1     Running   0             16h   10.50.0.5      aks-apps-40337985-vmss000000
monitoring    prometheus-monitoring-kube-prometheus-prometheus-0  2/2     Running   0             18h   10.244.0.108   aks-system-27949052-vmss000000
```

Figure 4: AKS node and pod output identifying system and application pools.

Terraform deployment and platform components

Helm provider/charts. Terraform can bootstrap selected cluster add-ons, while Helm charts encapsulate the installation settings for Argo CD, Grafana, Prometheus, and Loki. Their deployments are distinct from the Terraform provisioning of AKS itself; the report does not assume every chart is managed by the same Terraform module.

Layer	Separation of responsibilities
Azure resources / AKS	Terraform state, variables, network and cluster definitions
Node pools	System components versus application scheduling
Cluster platform	Helm-based deployment of GitOps and observability components
Application	Reusable Crewmeister Helm chart and GitOps revision

WORKSTREAM 03 / PACKAGING AND CONTINUOUS INTEGRATION

4. Docker, GHCR and DevSecOps CI

Dockerfile and Docker Compose. The Dockerfile packages the Spring Boot service as a portable image, while `compose.yaml` starts the local application dependencies for repeatable validation and debugging. The same image may then be used in Kubernetes without rebuilding it for each environment.

GitHub Container Registry (GHCR). GHCR is the intended image distribution point, providing image tags and digests that can be referenced from Helm/GitOps configuration. Published digests should be checked in the workflow/registry before describing a specific release as immutable.

```
bechi@bechir:~/crewmeister-app$ sudo docker compose ps
NAME                IMAGE                  COMMAND                  SERVICE   CREATED         STATUS                   PORTS
crewmeister-app     crewmeister-app:local  "java -jar /app/app.…"   app       9 minutes ago   Up 8 minutes (healthy)   127.0.0.1:8080->8080/tcp
crewmeister-mysql   mysql:8.4              "docker-entrypoint.s…"   mysql     9 minutes ago   Up 9 minutes (healthy)   3306/tcp, 33060/tcp
```

Figure 5: Local application containerization and Docker Compose evidence.

GitHub Actions: build, checks and security feedback

GitHub Actions. Workflow jobs automate Maven validation, security tooling, container packaging and publication. CI must be distinguished from deployment authorization: a green build alone does not approve a Kubernetes rollout.

Tool	Role	Behavior / evidence
Maven	Build and test	Compile, unit tests and package output
Gitleaks	Secret detection	Search committed content for leaked credentials
SonarQube Cloud	Code analysis	Quality and maintainability feedback; gate enforcement depends on configuration
Trivy	Vulnerability scan	**Non-blocking, report-only**: findings recorded without failing the delivery by design
Docker / GHCR	Package and publish	Traceable image tag or digest

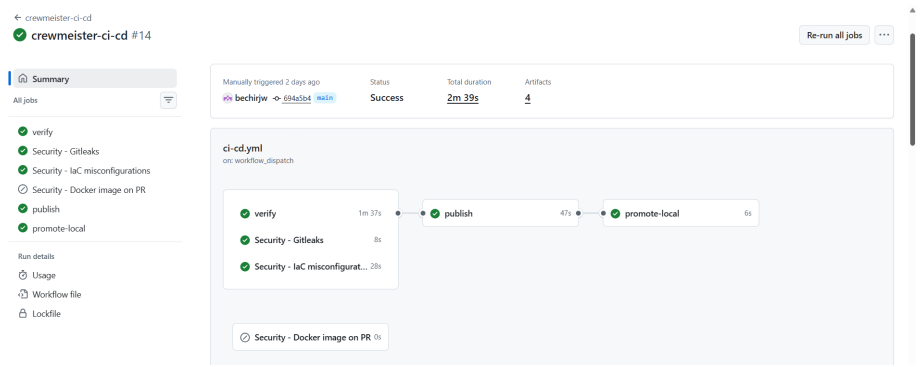


Figure 6: GitHub Actions workflow execution and job status.

Security caveat: a report-only Trivy stage is visibility, not a blocking security gate. Branch protection, required checks, and explicit fail conditions must be checked independently.

WORKSTREAM 04 / DECLARATIVE DELIVERY AND STORAGE

5. Helm, Argo CD and persistent storage

Helm. Helm organizes Kubernetes resources as configurable chart templates. The Crewmeister chart in `helm/crewmeister/` supports common deployment logic with environment-specific values instead of duplicated manifest sets.

Argo CD. Argo CD compares the desired revision in Git with running Kubernetes resources and synchronizes according to the application's configured policy. This is **GitOps CD**: infrastructure deployment is not initiated directly by an unrestricted CI push.

Change-controlled delivery workflow

1. Build/test and report security findings in GitHub Actions.
2. Publish a versioned application image to the container registry.
3. Update the Helm values or Argo CD desired revision through Git.
4. Merge an authorized change; Argo CD reconciles the target cluster.
5. Check application health, sync status and workload rollout.

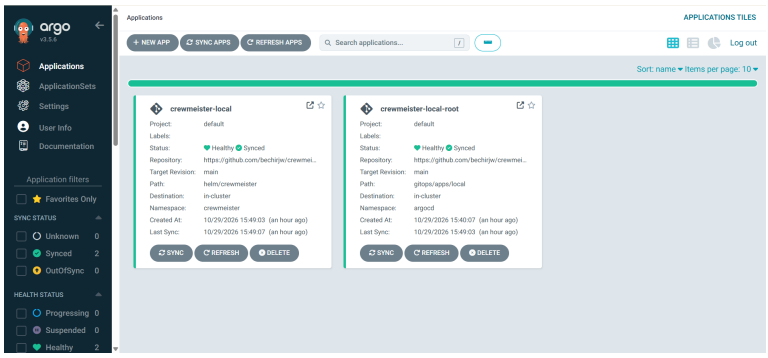


Figure 7: GitOps application and Kubernetes workload evidence from the local cluster.

Local Path Provisioner: dynamic lab storage

Local Path Provisioner. A Kubernetes StorageClass dynamically satisfies PersistentVolumeClaims (PVCs) using storage attached to an on-premises node. The deployment documents claims for stateful services, including MySQL and observability components.

WORKSTREAM 05 / OPERATIONS AND SECURITY

6. Observability, alerts and security

Prometheus. Prometheus collects time-series metrics for cluster and application monitoring. **Grafana.** Grafana visualizes the metrics, while dashboards support CPU, memory, pod, and workload diagnosis. **Loki.** Loki stores/query-indexes log streams so operators can investigate application and Kubernetes events together.

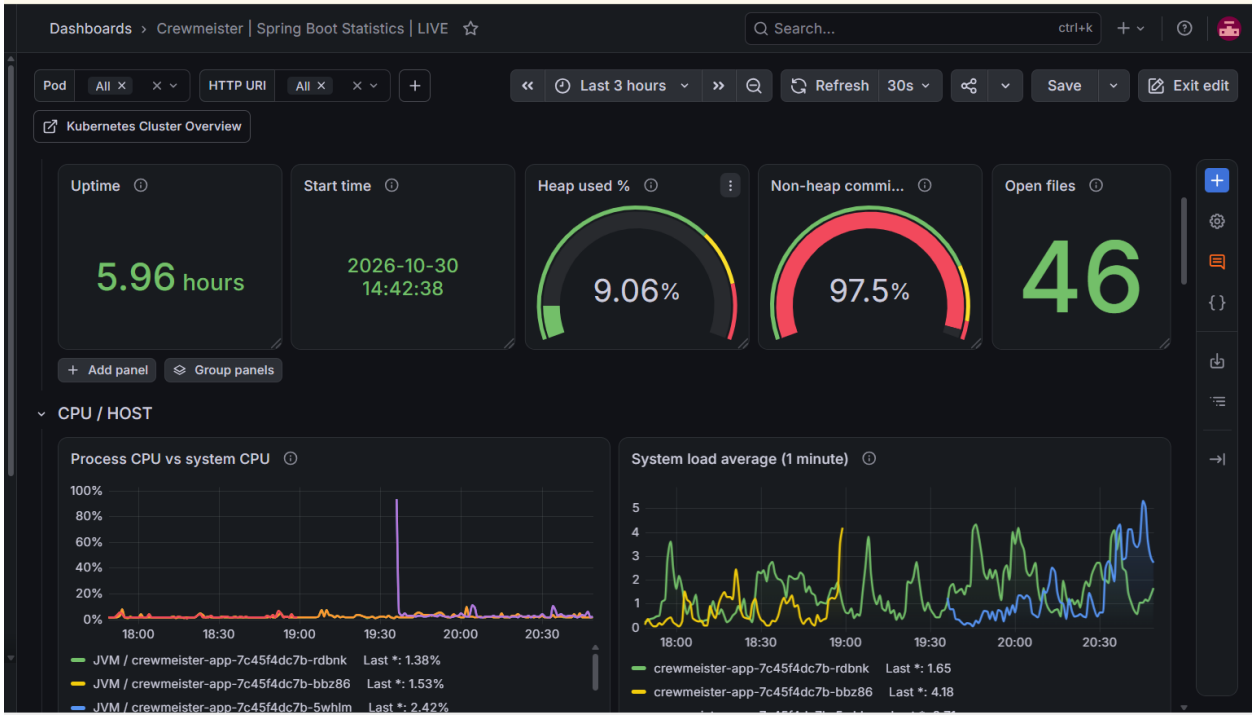


Figure 8: Observability evidence .

Alerting. Grafana Alerting or Alertmanager evaluates conditions and can dispatch notifications. The repository shows monitoring components; notification routing should be treated as verified only after a recorded alert firing and delivery test.

SCREENSHOT SLOT

monitoring-evidence-2.png

Copy the file from the repository's docs/screenshots folder

Figure 9: Additional monitoring/logging evidence referenced in the README.

Repository scanning and operational safeguards

GitHub security. Repository security capabilities can surface dependency alerts and secret/code-scanning findings. The CI adds Gitleaks, SonarQube Cloud and report-mode Trivy for complementary feedback. These tools differ in scope and must not be presented as equivalent blocking checks.

Recommended safeguard	Project implication
Least privilege and secrets	Protect GHCR tokens, cloud identity, kubeconfigs, and Terraform state
Review and provenance	Trace commits to CI results, container digest and GitOps changes
Operational resilience	Watch storage pressure, node health, alerts and service recovery

Conclusion

The project demonstrates a coherent DevSecOps path spanning **reusable Ansible automation, modular AKS provisioning, Docker images, CI security feedback, Helm/Argo CD GitOps, persistent storage, and observability**. The remaining hardening priorities are validating enforced policy gates, testing alert delivery and recovery, and documenting production-grade storage and secret controls.

Primary reference: `https://github.com/bechirjw/crewmeister-app`. Figures are sourced from repository screenshots when supplied; otherwise the LaTeX preview displays labeled evidence slots.